

La Tua Architettura Regge la Crescita?

Guida pratica per CTO e CEO tecnici che vogliono risposte, non teorie

di Elios Scoglio | 108 Vision

Con oltre 20 anni di lavoro su sistemi enterprise — piattaforme che vendono milioni di biglietti, che devono reggere picchi di centomila utenti in pochi secondi, che integrano legacy C++/CORBA con microservizi Java moderni — ho visto architetture reggere e architetture crollare.

Il costo di un'architettura che cede non è mai solo tecnico. E' costo operativo quotidiano, sviluppatori che se ne vanno, opportunità di business mancate, incidenti che bruciano la fiducia dei clienti.

In questo manuale trovi gli strumenti per diagnosticare dove sei, le domande giuste da fare prima di prendere decisioni costose, e un framework operativo per migliorare in modo incrementale senza riscrivere tutto.

Non troverai dogmi. Troverai criteri, esempi concreti, e risposte dirette — anche quando la risposta onesta è "dipende, e ti spiego da cosa".

1. I 5 Segnali che la Tua Architettura Sta Cedendo

L'architettura non crolla in modo drammatico. Cede lentamente, per accumulo. Come una barca che prende acqua da una dozzina di piccole crepe invece di un solo squarcio. I sintomi arrivano prima nei processi e nelle persone, e solo dopo nei sistemi.

Segnale 1 — Le Release Rallentano Mentre il Codice Cresce

All'inizio, un nuovo sviluppatore aggiungeva una feature in due giorni. Ora ci vogliono due settimane per la stessa complessità. Non perché il team sia peggiore — spesso e' piu' esperto. Ma ogni modifica richiede di capire un contesto enorme, verificare che non rompa qualcosa di inaspettato, attendere una pipeline di test che dura quaranta minuti.

Oppure: hai un componente che "tutti evitano". Un file da 3.000 righe che fa tutto. Ogni volta che lo tocchi, qualcosa si rompe da un'altra parte.

La causa sottostante e' l'assenza di bounded context. Il codice e' cresciuto senza che nessuno abbia definito con chiarezza quali concetti appartengono a quale modulo. Le dipendenze si moltiplicano in modo silenzioso. Il modello dati dell'"ordine" viene usato anche per generare report, inviare email, fare riconciliazione contabile. Questo e' accoppiamento nascosto: non e' esplicito nell'architettura, e' inciso nel corpo del codice.

[verificato] In sistemi di medie dimensioni, questo pattern e' quasi universale nella fase di crescita tra il lancio e i 50.000 utenti attivi mensili. E' la storia naturale di ogni prodotto che ha avuto successo senza un'architettura pensata per il cambiamento.

L'urgenza e' alta ma non critica nell'immediato. Il problema peggiora ogni mese e il costo di risolverlo cresce esponenzialmente. Cosa fare: mappare i bounded context esistenti, identificare le dipendenze cicliche, iniziare a isolare i moduli piu' volatili in modo incrementale.

Segnale 2 — Gli Incidenti Cascadeggiano

Un servizio esterno di pagamento va in timeout. Nel giro di trenta secondi il sistema di ordini e' irraggiungibile. Nel giro di due minuti l'intera applicazione non risponde. Il CEO chiama.

Eppure il problema originale era "solo" un timeout del gateway di pagamento.

La causa e' il cascading failure: un guasto in un punto si propaga a cascata perche' non ci sono meccanismi di isolamento. In un sistema che fa chiamate a servizi esterni, ogni chiamata sincrona senza timeout e' una potenziale bomba. Il circuit breaker e' la soluzione. Ma per applicarlo devi prima riconoscere che il problema esiste.

[verificato] Il pattern piu' comune che ho visto: chiamata sincrona a un sistema fiscale senza timeout configurato correttamente. Il sistema fiscale va in latenza elevata. Gli ordini si accumulano in attesa. Il pool di connessioni si esaurisce. Il gateway risponde 503 a tutto.

L'urgenza e' critica. Ogni incidente a cascata e' perdita di vendite, perdita di fiducia degli utenti, potenziale violazione di SLA. Se il tuo sistema non ha timeout configurati su ogni chiamata esterna, hai una bomba a orologeria.

Segnale 3 — Ogni Nuova Feature Tocca 5 File in 3 Moduli

Il PM chiede di aggiungere un campo "codice fiscale" nel profilo utente. Sembra semplice. Il developer deve modificare: il controller HTTP, il DTO di richiesta, il DTO di risposta, il service, il repository, il modello di dati, la migration SQL, la validazione, il mapper, i test. Undici modifiche per un campo.

La causa e' la violazione del Single Responsibility Principle a livello architetturale. Quando ogni layer ha la sua rappresentazione dello stesso concetto senza separazione chiara delle responsabilita', ogni modifica richiede una propagazione verticale su tutti i layer.

Spesso ci sono anche God classes: un OrderService da 4.000 righe che gestisce la creazione dell'ordine, la cancellazione, il rimborso, la spedizione, la notifica email, la generazione del PDF, la sincronizzazione con il CRM. Non e' un servizio — e' un monolite travestito da classe.

[probabile] In team di 5-15 sviluppatori che hanno lavorato sotto pressione di delivery per 2-3 anni, questo schema e' quasi inevitabile senza una governance architetturale attiva. Non e' colpa del team. E' la conseguenza naturale dell'assenza di confini espliciti.

Segnale 4 — Il Team Ha Paura di Fare Refactoring

"Non tocchiamo quel modulo — funziona." Questa frase e' il segnale piu' chiaro che qualcosa non va. In un'architettura sana, il refactoring e' una routine ordinaria. Se il team evita attivamente di toccare parti del sistema, e' perche' quelle parti non hanno una rete di protezione.

Due problemi spesso combinati: test coverage insufficiente sul codice critico, e assenza di documentazione delle decisioni architetturali.

Il codice senza test e' codice senza rete. E se hai solo unit test che mockano ogni dipendenza ma nessun test di integrazione che verifica il flusso reale, puoi avere 80% di coverage e zero fiducia reale.

L'assenza di Architecture Decision Record crea un secondo problema: la conoscenza vive nelle teste delle persone. Quando il developer che ha scritto quel modulo sei anni fa non lavora piu' in azienda, nessuno sa perche' le cose sono fatte in quel modo.

[verificato] Il costo reale della paura del refactoring si misura indirettamente: feature aggiunte "intorno" al codice cattivo invece che dentro, duplicazioni, branch di lunga durata, sviluppatori senior che evitano i ticket che toccano certi moduli.

Segnale 5 — Onboarding di un Nuovo Sviluppatore Richiede Settimane

Un nuovo senior developer entra in azienda. Ha vent'anni di esperienza. Dopo tre settimane non ha ancora fatto merge di una PR in produzione. Sta ancora cercando di capire dove vive la logica di business, perche' ci sono tre modi diversi di fare la stessa cosa, come funziona il deploy.

La causa e' la conoscenza tribale. La documentazione e' considerata un costo opzionale. Il risultato e' che la conoscenza vive nelle persone, non nel sistema.

Questo crea un rischio organizzativo reale: se Marco prende tre settimane di ferie, il sistema di pagamento diventa una black box. La mancanza di ADR, di README operativi, di diagrammi architetturali aggiornati non e' un problema estetico — e' un rischio operativo e un freno alla scalabilita' del team.

[verificato] Un buon indicatore: quanto tempo ci vuole a un nuovo developer per fare merge della sua prima PR? In sistemi ben documentati, questo tempo si misura in giorni. In sistemi con conoscenza tribale diffusa, sono settimane.

****Insight 108 Vision**** — I sintomi dell'architettura che cede arrivano prima nelle persone che nel codice. Se il team evita di toccare certi moduli, se il turnover e' alto, se l'onboarding dura mesi, hai gia' il segnale. Il codice viene dopo.

2. Monolite vs Microservizi: La Risposta Onesta

Partiamo da una premessa scomoda: la maggior parte degli articoli sui microservizi e' scritta da persone che lavorano in Netflix, Amazon o Spotify. Aziende con 500+ sviluppatori, team dedicati per ogni servizio, piattaforme di orchestrazione mature.

Se hai un team di 10 persone e stai pensando di "passare ai microservizi", stai leggendo la letteratura sbagliata.

Perche' i Microservizi Vengono Consigliati Sempre (e Perche' Spesso e' Sbagliato)

I microservizi hanno senso quando:

- Hai team indipendenti che possono deployare senza coordinarsi
- Hai bounded context chiari e stabili nel dominio
- Hai la maturita' operativa per gestire un sistema distribuito
- Il beneficio della scalabilita' indipendente vale il costo della complessita' aggiunta

I microservizi diventano un problema quando:

- I confini del dominio non sono chiari e stai cercando di imporli attraverso l'architettura
- Il team e' piccolo e deve coordinarsi comunque su ogni feature
- Non hai l'infrastruttura per gestire un sistema distribuito
- Stai migrando da un monolite accoppiato male e stai distribuendo l'accoppiamento, non eliminandolo

La domanda giusta prima di decidere e': "Ho team indipendenti con bounded context chiari nel dominio?"

Se la risposta e' no, i microservizi ti faranno quasi certamente del male nel breve-medio termine.

La Tabella Comparativa Onesta

Dimensione	Monolite modulare	Microservizi	Distributed monolith
Semplicita' operativa	Alta	Bassa	Molto bassa
Velocita' di sviluppo iniziale	Alta	Media	Molto bassa
Scalabilita' indipendente	No	Si (se ben fatto)	Apparente
Isolamento dei fallimenti	Parziale	Alto	Basso
Overhead infrastrutturale	Basso	Alto	Molto alto
Adatto per PMI/team piccoli	Si	Solo dopo scala significativa	Mai

Il Distributed Monolith: L'Incubo da Evitare

Il distributed monolith e' il risultato di aver decomposto un monolite in servizi separati senza aver prima identificato i bounded context corretti. E' peggio del monolite originale perche' ha tutti i problemi del monolite piu' tutti i problemi del sistema distribuito.

Come riconoscerlo:

- Deploy coordinato: non puoi deployare il servizio A senza deployare anche il servizio B
- Database condiviso: due o piu' servizi accedono alle stesse tabelle direttamente
- Comunicazione sincrona pervasiva: ogni operazione di business richiede chiamate a catena tra 4-5 servizi
- Team che si coordinano su ogni feature: una singola feature richiede modifiche sincronizzate su 3 team diversi

[verificato] Il segnale diagnostico piu' rapido: chiedi al team "puoi deployare questo servizio da solo senza toccare gli altri?". Se la risposta e' no con piu' di un paio di eccezioni, hai un distributed monolith.

Quando i Microservizi Hanno Senso: Criteri Oggettivi

- Team size: hai almeno 3 team di almeno 4 persone ciascuno, con ownership chiara di bounded context distinti
- Bounded context stabili: il tuo dominio ha confini identificabili. Se non riesci a disegnarli sulla lavagna senza che il team litighi, non sono abbastanza chiari
- Requisiti di scalabilita' differenziata reale: hai componenti che devono scalare in modo radicalmente diverso dagli altri. "Forse un giorno servirà" non e' un criterio
- Maturita' operativa: hai gia' logging strutturato, distributed tracing, health check, pipeline CI/CD automatica. Se non hai queste basi, la complessita' distribuita amplifichera' i problemi
- Competenze DevOps: qualcuno deve saper gestire orchestrazione container, service mesh, rollback su sistemi distribuiti

****Insight 108 Vision**** — Il modular monolith ben governato e' una strategia architettuale seria e spesso piu' moderna di microservizi accoppiati male. Non e' un ripiego — e' una scelta consapevole.

Come Migrare Senza Riscrivere Tutto: Lo Strangler Fig Pattern

Non riscrivere. Estrarre.

L'idea: aggiungi un facade/proxy davanti al sistema esistente. All'inizio il facade manda tutto al sistema originale. Gradualmente devia il traffico verso il nuovo servizio, incrementalmente.

Come funziona in pratica:

- Identifica un bounded context con valore autonomo — non il piu' semplice, quello che porta piu' beneficio se estratto
- Aggiungi un facade/proxy che controlla il traffico verso quella funzionalita'
- Scrivi il nuovo servizio con i test (prima i test di contratto, poi l'implementazione)
- Migra il traffico in modo incrementale: prima 1%, poi 10%, poi 50%, poi 100%
- Elimina il codice nel sistema originale solo quando il nuovo servizio regge il 100% del traffico per un periodo sufficiente

Questo approccio richiede mesi, non settimane. Ma riduce drasticamente il rischio rispetto a una riscrittura completa.

3. Tech Debt: Come Classificarlo, Misurarlo, Gestirlo

La Metafora Originale (Non la Versione Distorta)

Ward Cunningham, uno dei padri del movimento Agile, ha coniato il termine "technical debt" nel 1992. L'idea originale era precisa: a volte e' ragionevole scrivere codice non perfetto per andare in produzione piu' velocemente — come contrarre un debito per cogliere un'opportunita'. Il "debito" e' il costo futuro di migliorare quel codice.

Cunningham intendeva il debito come una scelta deliberata e consapevole, con un piano di rimborso. Nel corso degli anni, il termine e' diventato la parola per giustificare qualsiasi compromesso tecnico, inclusi quelli fatti per pigrizia o fretta.

I 4 Tipi di Tech Debt

Design debt: confini di modulo sbagliati, accoppiamenti stretti, God classes. Il piu' costoso nel lungo periodo perche' vincola ogni decisione futura.

Test debt: codice critico senza test, test che non verificano il comportamento reale, pipeline lenta che il team bypassa. Aumenta il costo di ogni modifica.

Infrastructure debt: dipendenze obsolete con vulnerabilita' di sicurezza, ambienti di sviluppo che non replicano la produzione, pipeline non automatizzate.

Knowledge debt: assenza di documentazione, architettura non documentata, processi operativi nella testa di una persona. Non e' codice — ma il suo costo e' reale quanto gli altri.

Come Misurare Senza Strumenti Costosi

Proxy per il design debt:

- Numero medio di file modificati per feature: sopra 5 e' un segnale
- Dimensione media delle classi principali: sopra 500 righe merita attenzione
- Dipendenze cicliche tra moduli
- Tempo medio per fare merge di una PR su feature standard

Proxy per il test debt:

- Coverage dei percorsi critici di business (non coverage generica)
- Numero di hotfix in produzione negli ultimi 6 mesi per regressioni
- Durata della pipeline CI: oltre 20 minuti e' un indicatore che i test vengono bypassati

Proxy per il knowledge debt:

- Numero di key man dependencies nel team
- Tempo medio di onboarding
- Quante volte al giorno vengono fatte domande a cui la documentazione dovrebbe rispondere

La Matrice Impatto/Sforzo

	BASSO SFORZO	ALTO SFORZO
ALTO IMPATTO	Quick wins fai subito	Progetti strategici pianifica con cura
BASSO IMPATTO	Fill-ins quando hai tempo	Evita o posponi valuta se vale

Quick wins: refactoring di una God class che viene toccata ogni settimana, aggiunta di test su un modulo critico gia' isolato, documentazione di un processo che si spiega a voce ogni volta.

Progetti strategici: decomposizione di un bounded context mal definito, migrazione da un ORM legacy, riscrittura di un

layer di integrazione critico con pattern corretti.

Evita o posponi: riscrivere un modulo stabile che funziona bene ma non segue le ultime convenzioni stilistiche. Il costo non giustifica il rischio.

Quando il Tech Debt e' un Rischio Esistenziale

Il tech debt diventa un rischio esistenziale quando:

- Paralizza le release: il team non riesce a fare una release minore senza rischio alto di regressioni
- Blocca l'assunzione: nuovi developer senior se ne vanno dopo tre mesi perche' non sopportano lavorare in quel codebase
- Crea vulnerabilita' di sicurezza: dipendenze obsolete con CVE note, nessun processo di aggiornamento
- Rende impossibile il disaster recovery: nessuno sa come ripartire il sistema dopo un failure completo

[verificato] Il segnale che preoccupa di piu' in una code review non e' la qualita' del codice — e' l'atteggiamento del team. Se gli sviluppatori senior sono demotivati, se nessuno propone miglioramenti, se le PR vengono fatte "solo per chiudere il ticket", il debito tecnico ha gia' eroso la cultura. Quello e' il costo piu' difficile da recuperare.

Come Parlare di Tech Debt al CEO/Board

Il CEO non si preoccupa della qualita' del codice. Si preoccupa dei costi, dei ricavi, del rischio. Devi tradurre.

"Abbiamo troppo tech debt" non dice nulla.

"Il 30% del tempo del team va in rework e bug causati da un'architettura che non e' mai stata aggiornata. Se non affrontiamo il problema nei prossimi 6 mesi, ogni nuova feature costera' il 40% in piu' del necessario." — questo dice qualcosa.

Le leve di traduzione:

- Velocita' di delivery: "Ogni sprint, 2 giorni su 10 vanno in debugging su codice legacy. Con 3 mesi di refactoring mirato, recuperiamo 1 giorno a sprint — permanentemente."
- Rischio operativo: "Il modulo di pagamento non ha test automatici su 4 dei 6 flussi critici. Ogni release su quel modulo e' un rischio concreto."
- Costo del personale: "Il tempo di onboarding medio e' 6 settimane. Con documentazione e architettura chiara, potremmo portarlo a 3."

***Insight 108 Vision** — Il tech debt non e' un problema tecnico che vuoi risolvere per soddisfazione personale. E' un costo operativo che stai gia' pagando, ogni giorno. Il lavoro e' renderlo visibile nel linguaggio del business.*

4. Resilienza: Come Non Cadere ai Picchi di Traffico

[verificato] Lavoro su sistemi di ticketing dove il traffico passa da zero a 100.000+ utenti attivi nel giro di secondi. Il sistema deve reggere il picco senza degrado percepibile dall'utente, mantenendo correttezza transazionale: nessun doppio booking, nessun pagamento perso.

Questo mi ha insegnato a riconoscere i pattern di fallimento prima che accadano.

I 3 Fallimenti Piu' Comuni Sotto Carico

Cascading failures: un componente va lento. I chiamanti aspettano. I loro thread si esauriscono. I loro chiamanti aspettano. Il fallimento si propaga verso l'alto. Il sistema non risponde non perche' il servizio sia morto, ma perche' e' diventato lento.

Timeout non configurati: il default di molti HTTP client e' "aspetta indefinitamente". In un sistema sotto carico, una chiamata che non risponde entro 30 secondi non rispondera' entro 100 secondi — ma nel frattempo ha consumato un thread e una connessione dal pool.

[verificato] Ho visto sistemi andare in crisi completa non perche' un database fosse morto, ma perche' era lento. La latenza si moltiplicava per il numero di richieste in attesa, esauriva i pool di connessione, e il sistema smetteva di rispondere a tutto.

Database bottleneck: query non indicizzate su tabelle cresciute di un ordine di grandezza, N+1 query, transazioni troppo lunghe che tengono lock per secondi, pool di connessioni troppo piccolo.

Circuit Breaker: Cos'e' e Perche' Serve

Il circuit breaker "apre" il circuito quando rileva un tasso di fallimenti superiore a una soglia configurata, rigettando le richieste immediatamente invece di farle aspettare timeout.

Stati:

- Chiuso (normale): le richieste passano. Il circuit breaker conta i fallimenti.
- Aperto (fallimento rilevato): dopo N fallimenti consecutivi, le richieste vengono rigettate immediatamente con un errore rapido e controllato.
- Half-open (test di recovery): dopo un timeout configurato, lascia passare una richiesta di prova. Se ha successo, torna chiuso.

Il beneficio principale: fallimento veloce invece di attesa indefinita. Invece di tenere il thread bloccato per 30 secondi aspettando una risposta che non arrivera', il circuit breaker risponde in millisecondi con un errore noto e gestibile.

In .NET si usa Polly, in Java si usa Resilience4j.

Timeout e Retry: La Regola Pratica

Ogni chiamata remota deve avere un timeout. Non un timeout "ragionevole" come 60 secondi — un timeout aggressivo calibrato su quello che il sistema puo' tollerare. Se l'utente si aspetta una risposta entro 3 secondi, le chiamate interne non possono avere timeout di 10 secondi.

Retry ha senso solo per errori transienti (rete momentaneamente non disponibile, 503 temporaneo). Non ha senso per errori permanenti (404, 400, 401). Il retry senza discernimento amplifica il problema.

Retry con exponential backoff + jitter: aspetta 100ms, poi 200ms, poi 400ms, con variazione casuale per evitare che tutti i client ritentino esattamente nello stesso momento.

Ordine corretto di composizione delle policy: Bulkhead (facoltativo) → Timeout → Retry → CircuitBreaker.

Idempotenza: Perche' E' Non Negoziabile

Un'operazione e' idempotente quando eseguirla N volte produce lo stesso risultato di eseguirla una volta sola.

In un sistema distribuito con retry, e' inevitabile che alcune operazioni vengano eseguite piu' di una volta. La rete puo' interrompere la risposta dopo che l'operazione e' stata eseguita ma prima che il chiamante riceva conferma.

Se l'operazione non e' idempotente, il risultato e' un pagamento addebitato due volte, un biglietto emesso due volte, un'email inviata due volte.

Come si realizza: l'approccio piu' semplice e' un idempotency key — un identificativo univoco della richiesta generato dal client. Il server memorizza le operazioni gia' eseguite per quella chiave e, se riceve la stessa richiesta due volte, risponde con il risultato della prima esecuzione senza rieseguire.

[verificato] L'idempotenza sul flusso di acquisto e' non negoziabile. Un doppio booking o un doppio addebito e' un incidente con impatto legale ed economico diretto.

Cache: Quando Aiuta e Quando Nasconde i Problemi

La cache aiuta genuinamente quando:

- I dati cambiano raramente ma vengono letti spesso
- I risultati di computazioni costose possono essere riutati
- Si riduce il carico sul database per query identiche ripetute in burst

La cache nasconde i problemi quando:

- Viene usata per compensare query lente che andrebbero ottimizzate
- La gestione dell'invalidazione e' complessa e buggy
- Diventa un layer critico: se la cache cade, il sistema e' irraggiungibile

Regola empirica: misura prima di aggiungere cache. Se una query impiega 5ms, la cache non serve. Se impiega 500ms, prima ottimizza la query; se non riesci a portarla sotto 50ms, valuta allora la cache.

****Insight 108 Vision**** — La resilienza non si aggiunge alla fine. Si progetta dall'inizio. Ogni chiamata remota senza timeout e' un atto di ottimismo ingiustificato. Ogni operazione critica senza idempotenza e' un bug che non e' ancora emerso.

5. Observability: Come Sapere Cosa Sta Succedendo in Produzione

In produzione non puoi usare il debugger. Log, metriche e trace sono gli unici strumenti reali.

I 4 Golden Signals

Codificati da Google nel Site Reliability Engineering book. Quattro metriche che, monitorate insieme, danno una visione completa della salute di un sistema.

Latenza: quanto tempo ci vuole per rispondere a una richiesta. Non monitorare solo la media — e' bugiarda. Monitora P95 e P99. Se il P99 e' 2 secondi con 10.000 utenti al minuto, sono 100 utenti al minuto con un'esperienza degradata.

Traffico: quante richieste ricevi per unita' di tempo. Fondamentale non solo per il capacity planning ma per diagnosticare problemi: se gli errori aumentano ma il traffico cala, il problema e' probabilmente interno.

Errori: percentuale di richieste che terminano con errore. Distingui: errori 4xx (il client sta sbagliando) e errori 5xx (il tuo sistema ha un problema). Un aumento del tasso di 5xx e' sempre un alert da investigare.

Saturazione: quanto pieno e' il tuo sistema. CPU, memoria, pool di connessioni al database, profondita' delle code. La saturazione predice i problemi prima che si manifestino come errori.

La Differenza tra Log, Metrics e Traces

Log: eventi discreti con contesto. "L'ordine 12345 e' stato creato dall'utente 678 alle 14:32." Utili per diagnosticare problemi specifici. La regola fondamentale: log strutturati in JSON, mai in formato free text. Include sempre un trace_id per correlare gli eventi di una singola richiesta.

Metrics: valori numerici aggregati nel tempo. Non hanno contesto individuale ma permettono di vedere trend, anomalie, soglie. Sono economiche da raccogliere. Sono lo strumento principale per il monitoring operativo.

Traces: il percorso di una singola richiesta attraverso il sistema distribuito. "La richiesta ha chiamato il gateway (23ms), che ha chiamato il servizio ordini (45ms), che ha chiamato il database (8ms)." Indispensabili per diagnosticare dove la latenza si accumula. OpenTelemetry e' lo standard de facto.

Alert che Funzionano: Come Evitare Alert Fatigue

L'alert fatigue e': il sistema invia cosi' tanti alert che il team smette di leggerli. Quando succede, l'alert perde ogni utilita' — compreso quello che annuncia un incidente reale.

Regola fondamentale: ogni alert deve richiedere un'azione. Se ricevi un alert e non sai cosa fare, o se la risposta giusta e' "non fare niente, si risolve da solo", quell'alert non dovrebbe esistere.

Alert da avere sempre:

- Tasso di errori 5xx sopra soglia per un periodo sostenuto (es: > 1% per 5 minuti)
- Latenza P99 sopra soglia per endpoint critici (es: > 2 secondi per 3 minuti)
- CPU/memoria sopra soglia di saturazione (es: > 85% per 10 minuti)
- Health check che fallisce
- Tasso di transazioni che scende a zero durante un'operazione attesa

Alert da evitare:

- Alert su picchi momentanei senza persistenza temporale
- Alert su metriche intermedie che non impattano direttamente l'utente
- Alert duplicati sulla stessa condizione da sistemi diversi

Post-Mortem Blameless: Come Imparare dagli Incidenti

Un post-mortem blameless ha una premessa fondamentale: gli incidenti non succedono per colpa di singoli individui. Succedono perche' il sistema — processi, strumenti, architettura, comunicazione — ha creato le condizioni per cui un errore umano era probabile.

Se il post-mortem finisce con "Mario ha fatto un errore nel deploy", non hai imparato niente. Se finisce con "il processo di deploy non aveva una fase di verifica automatica dello schema del database prima dell'applicazione della migration", hai identificato qualcosa che puoi correggere.

Struttura minima di un post-mortem:

- Timeline: cronologia degli eventi dall'inizio del problema alla risoluzione, con timestamp precisi
- Impact: quanti utenti coinvolti, per quanto tempo, perdita stimata
- Root cause: causa tecnica profonda (5 whys applicato)
- Contributing factors: condizioni che hanno reso possibile il problema
- Action items: modifiche concrete con owner e deadline specifici
- What went well: cosa ha funzionato nel rilevamento e nella risposta

[verificato] I team che fanno post-mortem blameless regolari — anche per incidenti piccoli — migliorano la loro capacita' di risposta in modo misurabile nel tempo. E' il meccanismo principale con cui un team impara dagli errori invece di ripeterli.

6. Cloud e On-Premise: La Decisione con Criteri Reali

Il Mito "Il Cloud E' Sempre Meglio"

Il cloud ha cambiato radicalmente l'industria. Ma non e' gratuito, non e' automaticamente piu' sicuro, e non risolve i tuoi

problemi architetturali.

Se hai un monolite mal strutturato con tech debt elevato, il cloud ti dara' un monolite mal strutturato con tech debt elevato che costa di piu' e gira su istanze che non capisci completamente.

Criteri di Decisione Oggettivi

Criterio | Favorisce on-premise | Favorisce cloud

Variabilita' del carico | Carico prevedibile e stabile | Picchi imprevedibili, forte variazione stagionale

Dati sensibili | Normative stringenti, residenza obbligatoria | Dati standard con buona governance cloud

Competenze team | Forti in amministrazione tradizionale | Competenze cloud/DevOps presenti

Disaster recovery | Investimento infrastrutturale sostenibile | DR come servizio nativo preferito

I 3 Casi in Cui la Migrazione Ha Senso

Variabilita' del traffico estrema: se il carico varia di un ordine di grandezza tra minimo e massimo, il cloud permette di pagare solo per quello che si usa invece di over-provisionare permanentemente per il picco.

Disaster recovery e disponibilita' multi-region: costruire una DR geograficamente distribuita on-premise e' costoso. Il cloud la rende accessibile. Se il business richiede RPO/RTO bassi e non hai budget per infrastruttura DR dedicata, il cloud e' spesso la scelta piu' ragionevole.

Riduzione dell'overhead operativo per team piccoli: un team di 5 persone che gestisce anche l'infrastruttura paga un costo opportunita' elevato. Database managed, container orchestration managed, monitoring as a service — liberano tempo per lavoro a piu' alto valore.

Kubernetes: Quando Serve Davvero

Kubernetes e' eccellente per quello per cui e' stato progettato: orchestrare grandi flotte di container con requisiti complessi di scheduling, scaling, self-healing e deployment.

Non serve se:

- Hai meno di 10 servizi da gestire
- I tuoi requisiti di scaling sono gestibili con soluzioni piu' semplici
- Non hai un team con competenze Kubernetes
- Stai aggiungendo complessita' operativa senza un caso d'uso specifico che la giustifichi

[probabile] Per la maggior parte delle PMI italiane con team tecnici sotto 20 persone, Kubernetes e' over-engineering a meno che ci sia un caso d'uso specifico. La semplicita' operativa ha valore economico reale.

7. Come Fare un'Architecture Review: Il Piano d'Azione

Autodiagnosi in 30 Minuti: 5 Domande

Domanda 1: Quanto tempo ci vuole per fare una release in produzione?

Se supera 2-3 giorni per una feature standard, o richiede coordinamento tra piu' di 2 team, hai un problema di accoppiamento o di pipeline.

Domanda 2: Quante volte al mese avete un incidente in produzione?

Piu' di 2-3 incidenti al mese per un sistema maturo e' un segnale. La tipologia degli incidenti e' piu' informativa della frequenza: se si ripetono gli stessi tipi, il problema e' sistemico.

Domanda 3: Per aggiungere un campo a un'entity centrale, quanti file dovrete modificare?

Piu' di 7-8 file e' un segnale di accoppiamento verticale eccessivo. Piu' di 15 e' un problema serio.

Domanda 4: C'e' una persona la cui partenza bloccherebbe la vostra capacita' di fare release su un sistema critico?

Se la risposta e' si', avete una key man dependency che e' un rischio operativo e di retention.

Domanda 5: Avete un runbook per il disaster recovery — e vi siete esercitati su di esso negli ultimi 6 mesi?

"Abbiamo i backup" non e' un piano di DR. Un piano di DR descrive passo per passo come ripristinare il sistema, chi fa cosa, in quanto tempo, con quali criteri di verifica.

Checklist Pre-Review: 20 Domande Chiave

Architettura e Design

- ☐ I bounded context del dominio sono documentati e rispettati nel codice
- ☐ Le dipendenze tra moduli/servizi sono esplicite (diagramma aggiornato)
- ☐ Non esistono dipendenze cicliche tra moduli core
- ☐ Non ci sono God classes con responsabilita' non coese
- ☐ Le integrazioni con sistemi esterni sono incapsulate (Adapter o Anti-Corruption Layer)

Resilienza e Operativita'

- ☐ Ogni chiamata HTTP verso sistemi esterni ha un timeout configurato
- ☐ Ogni integrazione critica ha circuit breaker configurato
- ☐ I retry sono configurati con exponential backoff e jitter
- ☐ Le operazioni critiche (pagamento, prenotazione, emissione) sono idempotenti
- ☐ Esiste un runbook per il disaster recovery, testato negli ultimi 6 mesi

Observability

- ☐ I log sono strutturati in JSON e includono trace_id per correlazione
- ☐ Nessun dato sensibile e' presente nei log
- ☐ I 4 Golden Signals sono monitorati per ogni servizio critico
- ☐ Ogni servizio espone un health check funzionale
- ☐ Gli alert attivi richiedono tutti un'azione concreta

Testing e Qualita'

- ☐ I flussi critici di business hanno test di integrazione o E2E
- ☐ La pipeline CI/CD e' completamente automatizzata
- ☐ Il processo di deploy e rollback e' documentato e testato

Conoscenza e Documentazione

- ☐ Le decisioni architetturali significative degli ultimi 2 anni sono documentate (ADR)
- ☐ Non esistono key man dependencies su sistemi critici (ogni sistema critico ha almeno 2 persone che lo conoscono)

****Insight 108 Vision**** — Una buona architecture review non dice "avete fatto tutto male". Dice "questo funziona bene per le vostre dimensioni attuali; questo e' un rischio che cresce con la scala; questo e' un freno che potete risolvere in 3 mesi con X impegno". Il rispetto per il lavoro gia' fatto e' parte dell'analisi onesta.

8. Template ADR Minimale

Gli ADR (Architecture Decision Records) sono il meccanismo piu' efficace per preservare il "perche'" delle scelte

architetture. Un ADR compilato in 20 minuti e' infinitamente piu' utile di uno mai scritto perche' il template sembrava troppo formale.

```
# ADR-NNN: [Titolo della Decisione]
```

```
Data: YYYY-MM-DD
```

```
Stato: Proposta | Accettata | Sostituita da ADR-NNN | Deprecata
```

```
Deciders: [Chi ha partecipato alla decisione]
```

```
## Contesto
```

```
[Cosa sta succedendo che richiede questa decisione?
```

```
Includi i vincoli rilevanti: tecnici, di business, di tempo.]
```

```
## Decisione
```

```
[Cosa abbiamo deciso di fare? Inizia con "Abbiamo deciso di..."]
```

```
## Alternative considerate
```

```
### Alternativa 1: [Nome]
```

```
- Descrizione: ...
```

```
- Motivo del rifiuto: ...
```

```
### Alternativa 2: [Nome]
```

```
- Descrizione: ...
```

```
- Motivo del rifiuto: ...
```

```
## Conseguenze
```

```
Positive:
```

```
- [Beneficio atteso 1]
```

```
Negative / Trade-off:
```

```
- [Costo o rischio accettato]
```

```
Da monitorare:
```

```
- [Condizione che potrebbe cambiare questa decisione in futuro]
```

9. Glossario Essenziale

ADR (Architecture Decision Record): documento che registra una decisione architeturale significativa — contesto, alternative, decisione, conseguenze. Serve a preservare il "perche'" nel tempo. Un'architettura senza ADR diventa folklore nel giro di 18-24 mesi.

Bounded Context: il perimetro all'interno del quale un modello di dominio ha una definizione coerente. "Utente" nel checkout ha attributi diversi da "Utente" nel CRM. Riconoscere i bounded context e' il prerequisito per microservizi con confini corretti.

Circuit Breaker: pattern di resilienza che apre il circuito quando rileva fallimenti superiori a una soglia, rigettando le richieste immediatamente invece di aspettare timeout. Previene i cascading failures.

Distributed Monolith: anti-pattern architetture. Un sistema composto da piu' servizi separati che condividono database, hanno dipendenze di deploy strettamente accoppiate, o comunicano in modo sincrono pervasivo. Ha tutti i costi del sistema distribuito senza i benefici dell'isolamento.

Fitness Function: test automatizzato che verifica che un principio architeturale sia rispettato nel codice. Rende i principi

verificabili anziché solo dichiarati.

Golden Signals: i quattro segnali identificati da Google SRE come fondamentali per il monitoring: latency, traffic, errors, saturation.

Idempotenza: proprietà di un'operazione per cui eseguirla N volte produce lo stesso risultato di eseguirla una volta sola. Fondamentale in sistemi con retry automatici.

Strangler Fig Pattern: strategia di migrazione graduale da un sistema legacy. Si aggiunge un facade davanti al sistema originale; gradualmente le funzionalità vengono reimplementate nel nuovo sistema e il traffico deviato verso di esse.

Tech Debt: compromessi tecnici deliberati fatti per accelerare il delivery, con l'impegno implicito di migliorare il codice in seguito. Nel senso distorto, qualsiasi codice di bassa qualità, indipendentemente dal fatto che sia stato una scelta consapevole.

Note sui marcatori di confidenza usati in questo manuale:

- [verificato] — esperienza diretta, codice letto, metriche misurate
- [probabile] — inferenza basata su pattern riconosciuti, non verifica diretta su questo caso specifico
- Affermazioni senza marcatore riflettono conoscenza consolidata da letteratura tecnica riconosciuta

Vuoi andare oltre?

Vuoi applicare questo metodo alla tua azienda? Prenota 30 minuti con noi su 108vision.it — gratuito, senza impegno.

108 Vision — Costruiamo la direzione, non solo il codice.